F01 Account 必须理解的知识

> 这份文档不是让你死记代码，而是帮助你回答三个问题：一次请求从哪里进来、经过了谁、最后为什么得到这个结果。
>
> F01 一共实现四个功能：注册、登录、获取当前用户、修改密码。建议一边看本文，一边在 IDEA 中打开对应 Java 文件。

1. 1. 先建立最简单的整体认识

浏览器、Postman 或 IDEA HTTP Client 都属于“客户端”。Spring Boot 程序属于“服务端”。H2 或 MySQL 属于“数据库”。

一次普通请求的基本方向是：

```
客户端发送 HTTP 请求
        ↓
Controller 接收请求
        ↓
DTO 保存输入参数
        ↓
Service 定义要做什么
        ↓
ServiceImpl 实现业务规则
        ↓
Mapper 访问数据库
        ↓
Entity 对应数据库表
        ↓
VO 选择允许返回的数据
        ↓
ApiResponse 包成统一 JSON
        ↓
客户端收到响应
```

Fence 1

需要登录的请求会多经过一个 Interceptor：

```
客户端发送带 Token 的请求
        ↓
AuthenticationInterceptor 先验证 Token
        ↓
验证成功，把当前用户放入当前请求
        ↓
Controller 才会执行
```

Fence 2

你需要先记住：

- Controller 负责接请求，不应该堆业务逻辑。
- DTO 负责接收输入。
- Service 是业务接口。
- ServiceImpl 是业务的真正实现。

- Mapper 负责操作数据库。
- Entity 对应数据库表的一行。
- VO 负责返回安全的数据。
- Interceptor 在 Controller 前面执行。
- Utils 放可复用工具，本包的 JWT 生成和解析就在这里。

2. 2. 推荐阅读顺序与文件清单

2.1 2.1 先读四个核心文件

顺序	文件	类型	重点
1	`controller/AccountController.java`	Controller	四个接口从哪里进入
2	`service/AccountService.java`	Service	四个业务方法如何定义
3	`service/impl/AccountServiceImpl.java`	ServiceImpl	注册、登录、查询、改密的实际逻辑
4	`interceptor/AuthenticationInterceptor.java`	Interceptor	`/me` 和改密如何验证登录

2.2 2.2 再读输入和输出

文件	类型	用途
`dto/RegisterDTO.java`	DTO	接收注册的账号、密码、邮箱、手机
`dto/LoginDTO.java`	DTO	接收登录账号和密码
`dto/ResetPasswordDTO.java`	DTO	接收当前密码和新密码
`dto/CurrentUserDTO.java`	DTO	后端内部传递当前用户，不由客户端提交
`dto/TokenDTO.java`	DTO	JwtUtils 生成 Token 后的内部结果
`dto/TokenUserDTO.java`	DTO	从 Token 解析出的用户 id、版本、角色
`vo/AccountVO.java`	VO	返回脱敏账号资料
`vo/LoginVO.java`	VO	返回 Token、过期时间和账号资料

2.3 2.3 再读数据库和配置

文件	类型	用途
`entity/UserAccountEntity.java`	Entity	对应 `user_account` 表

文件	类型	用途
`mapper/UserAccountMapper.java`	Mapper	继承 MyBatis-Plus 的增删改查能力
`config/AccountConfig.java`	Config	创建 BCrypt 和 Clock Bean
`config/WebMvcConfig.java`	Config	指定哪些接口必须经过登录拦截器
`utils/JwtUtils.java`	Utils	生成、验证和解析 JWT

2.4 2.4 Account 包外仍然必须理解的文件

文件	为什么有关
`common/api/ApiResponse.java`	所有接口的统一 JSON 外壳
`common/exception/BusinessException.java`	表示可预期的业务错误
`common/exception/GlobalExceptionHandler.java`	把 Java 异常转换成 HTTP JSON 响应
`src/main/resources/schema.sql`	创建 `user_account` 表和唯一约束
`src/main/resources/application.yml`	H2、端口、JWT 密钥和有效期配置
`src/main/resources/application-mysql.yml`	启用 mysql profile 后使用的 MySQL 配置
`src/test/.../account/AccountApiTest.java`	完整验证四条链路
`http/account.http`	IDEA 中手工发送请求的样例

3. 3. 四个功能的总表

功能	HTTP	是否需要 Token	输入	输出
注册	`POST /api/accounts/register`	否	RegisterDTO	AccountVO
登录	`POST /api/accounts/login`	否	LoginDTO	LoginVO
获取当前用户	`GET /api/accounts/me`	是	CurrentUserDTO 由拦截器产生	AccountVO
修改密码	`POST /api/accounts/reset-password`	是	CurrentUserDTO + ResetPasswordDTO	LoginVO

`POST` 通常表示提交或修改数据，通常需要 JSON Body。`GET` 通常表示查询数据，一般不需要 Body。

4. 4. 功能一：注册的完整链路

4.1 4.1 客户端发送什么

```
POST http://localhost:80/api/accounts/register
Content-Type: application/json

{
  "account": "AAABBB",
  "password": "AAA123456",
  "email": "123@11.com",
  "mobile": "12333000120"
}
```

Fence 3

这些字段必须放在 Postman 的 `Body -> raw -> JSON` 中，不能放在 Params。因为 Controller 使用的是 `@RequestBody`。

4.2 4.2 Controller 如何接住请求

```
@PostMapping("/register")
@ResponseStatus(HttpStatus.CREATED)
public ApiResponse<AccountVO> register(
        @Valid @RequestBody RegisterDTO registerDTO) {
    return ApiResponse.success(accountService.register(registerDTO));
}
```

Fence 4

逐项理解：

- `@PostMapping("/register")`：这个方法只处理 POST 注册请求。
- 类上的 `@RequestMapping("/api/accounts")` 和这里的 `/register` 拼成完整路径。
- `@RequestBody`：把 JSON 请求体转换成 RegisterDTO。
- `@Valid`：转换完成后执行 DTO 上的校验注解。
- `@ResponseStatus(CREATED)`：成功状态码使用 201。
- `ApiResponse<AccountVO>`：统一响应中的 `data` 是 AccountVO。

4.3 4.3 RegisterDTO 做什么

RegisterDTO 使用 Java `record`：

```
public record RegisterDTO(String account, String password, String email,
String mobile) {
}
```

Fence 5

可以暂时把 record 理解成“专门保存数据的简化类”。Java 会自动提供：

```
registerDTO.account();
registerDTO.password();
registerDTO.email();
registerDTO.mobile();
```

Fence 6

校验注解的意思：

- `@NotBlank`：不能为 null、空字符串或只有空格。
- `@Size(min = 8, max = 72)`：密码长度 8～72。
- `@Email`：必须符合邮箱基本格式。
- `@Pattern`：必须符合正则表达式。

例如账号正则：

```
[A-Za-z0-9_]{4,32}
```

Fence 7

意思是：只能使用大小写字母、数字、下划线，总长度 4～32。

4.4 4.4 ServiceImpl 如何完成注册

Controller 调用：

```
accountService.register(registerDTO)
```

Fence 8

实际执行的是 AccountServiceImpl，因为它使用 `implements AccountService` 实现了接口，并且有 `@Service`。

注册过程分为以下步骤：

```
1. 账号去除前后空格并转为小写
2. 邮箱去空格并转为小写
3. 空手机号转换成 null
4. 查询账号、邮箱、手机号是否已存在
5. 创建 UserAccountEntity
6. BCrypt 加密密码
7. 设置角色、状态、Token 版本和时间
8. Mapper 执行 INSERT
9. Entity 转换成 AccountVO
10. Controller 返回 201 JSON
```

Fence 9

为什么账号 `AAABBB` 最后会变成 `aaabbb`：

```
value.trim().toLowerCase(Locale.ROOT)
```

Fence 10

- `trim()` 删除字符串前后空格。
- `toLowerCase()` 转成小写。
- 这样 `AAABBB` 和 `aaabbb` 不会注册成两个不同账号。

密码不是直接保存：

```
entity.setPasswordHash(passwordEncoder.encode(registerDTO.password()));
```

Fence 11

数据库里保存的是类似下面的 BCrypt 哈希：

```
$2a$10$......
```

Fence 12

哈希不是加密后的明文，不能“解密回原密码”。登录时只能用 `matches()` 判断输入密码是否匹配。

4.5 4.5 为什么查重做了两层

第一层是 Java 代码提前查询，目的是返回容易理解的提示：

```
账号已存在
邮箱已存在
手机号已存在
```

Fence 13

第二层是数据库 UNIQUE 约束：

```
CONSTRAINT uk_user_account_account UNIQUE (account)
```

Fence 14

原因是两个注册请求可能同时到达。它们都可能在提前查询时看到“尚未存在”，但数据库唯一约束仍能阻止重复写入。

4.6 4.6 注册为什么使用事务

```
@Transactional
```

Fence 15

可以先理解为：方法内的数据库操作要么全部成功，要么全部撤销。发生异常时 Spring 会回滚事务，不留下半条数据。

5. 5. 功能二：登录的完整链路

5.1 5.1 客户端请求

```
POST http://localhost:80/api/accounts/login
Content-Type: application/json

{
  "account": "AAABBB",
  "password": "AAA123456"
}
```

Fence 16

5.2 5.2 登录链路

```
JSON
  ↓
LoginDTO
  ↓
AccountController.login()
  ↓
AccountService.login()
  ↓
AccountServiceImpl.login()
  ↓
Mapper 根据标准化账号查 user_account
  ↓
BCrypt matches 校验密码
  ↓
检查账号是否 BANNED
  ↓
JwtUtils.createToken()
  ↓
LoginVO
  ↓
ApiResponse<LoginVO>
```

Fence 17

查询账号使用 MyBatis-Plus：

```
new LambdaQueryWrapper<UserAccountEntity>()
        .eq(UserAccountEntity::getAccount, account)
        .last("LIMIT 1")
```

Fence 18

大致等价于：

```
SELECT *
FROM user_account
WHERE account = ?
LIMIT 1;
```

Fence 19

密码校验：

```
passwordEncoder.matches(输入的明文密码, 数据库中的BCrypt哈希)
```

Fence 20

注意参数顺序：第一个是用户输入的明文，第二个是数据库哈希。

登录错误统一返回“账号或密码错误”，不告诉客户端究竟是账号不存在还是密码错误。这可以减少攻击者批量探测有效账号的机会。

5.3 5.3 登录成功并不等于服务端记住了你

当前项目没有传统 Session 登录。登录成功只是服务端签发一个 JWT 并交给客户端。

```
登录成功
   ↓
服务端生成 JWT
   ↓
Postman/前端保存 JWT
   ↓
以后每个需要登录的请求都主动携带 JWT
```

Fence 21

如果 Postman 没有在下一次请求里携带 Token，服务端就不知道你刚刚登录过。

6. 6. JWT 到底是什么，如何行动

6.1 6.1 JWT 的三个部分

JWT 看起来像：

```
xxxxx.yyyyy.zzzzz
```

Fence 22

三段分别是：

```
Header.Payload.Signature
头部.载荷.签名
```

Fence 23

- Header：记录 Token 类型和签名算法。
- Payload：记录用户 id、角色、版本、签发时间、过期时间。
- Signature：服务器用密钥计算的签名，用于发现篡改。

JWT 的 Header 和 Payload 只是 Base64URL 编码，不是加密。别人可以看到其中内容，所以不能把密码、身份证号等秘密放进 JWT。

6.2 6.2 当前 JWT 放了什么

JwtUtils 创建 Token 时写入：

```
.subject(entity.getId().toString())
.claim("role", entity.getRole())
.claim("ver", entity.getTokenVersion())
.issuedAt(Date.from(issuedAt))
.expiration(Date.from(expiresAt))
.signWith(key)
```

Fence 24

对应含义：

JWT 字段	含义
`sub`	用户 id
`role`	用户角色
`ver`	Token 版本
`iat`	签发时间
`exp`	过期时间

配置中的：

```
security:
  jwt:
    expiration-seconds: 7200
```

Fence 25

表示 Token 两小时后过期。

响应中的时间如果以 `Z` 结尾，表示 UTC 时间。中国标准时间是 UTC+8，不是少了八小时。

6.3 6.3 签名能防什么

假设用户把 Payload 中的角色从 USER 手动改成 ADMIN。Payload 内容改变后，原签名就不匹配。服务端用自己的 secret 验证签名时会失败，并返回 `INVALID_TOKEN`。

所以：

- 客户端可以读取 JWT 内容。
- 客户端不能在不知道 secret 的情况下合法修改内容。
- secret 必须保存在服务端，不能发送给前端。

6.4 6.4 JWT 如何跟着请求走

客户端需要发送请求头：

```
Authorization: Bearer eyJhbGciOi...
```

Fence 26

- `Authorization` 是请求头名称。
- `Bearer` 表示后面携带的是访问凭证。
- `Bearer` 后面必须有一个空格。
- Token 本身不能带双引号。
- Postman 选择 Bearer Token 后，只粘贴 `eyJ...`，Postman 会自动添加 `Bearer`。

6.5 6.5 JwtUtils 如何验证 Token

```
Jwts.parser()
        .verifyWith(key)
        .clock(() -> Date.from(clock.instant()))
        .build()
        .parseSignedClaims(token)
```

Fence 27

这个过程会验证：

1. Token 格式是否正确。
2. 签名是否正确。
3. Token 是否已经过期。

验证成功后读取 `sub`、`ver`、`role`，组成 TokenUserDTO。

6.6 6.6 为什么有 JWT 还要查询数据库

Token 只能证明“这个 Token 是服务器签发且没有被篡改”。但用户状态可能已经变化，例如：

- 账号被删除。
- 账号被禁用。
- 用户修改了密码。
- 角色被管理员调整。

所以拦截器仍然用 Token 中的 userId 查询最新数据库状态。这是一种更安全但每次会访问数据库的做法。

6.7 6.7 tokenVersion 如何让旧 Token 失效

注册时：

```
数据库 token_version = 0
JWT ver = 0
```

Fence 28

修改密码后：

```
数据库 token_version = 1
新 JWT ver = 1
旧 JWT ver 仍然是 0
```

Fence 29

拦截器比较：

```
entity.getTokenVersion().equals(tokenUserDTO.tokenVersion())
```

Fence 30

旧 Token 的 0 不等于数据库的 1，因此旧 Token 即使还没到 `exp`，也会返回 `TOKEN_REVOKED`。

7. 7. 功能三：获取当前用户的完整链路

7.1 7.1 客户端请求

```
GET http://localhost:80/api/accounts/me
Authorization: Bearer eyJhbGciOi...
```

Fence 31

这个请求没有 JSON Body。

7.2 7.2 为什么先经过拦截器

WebMvcConfig 注册了：

```
registry.addInterceptor(authenticationInterceptor)
        .addPathPatterns(
                "/api/accounts/me",
                "/api/accounts/reset-password");
```

Fence 32

因此 `/me` 的顺序不是直接进入 Controller，而是：

```
GET /me
  ↓
AuthenticationInterceptor.preHandle()
  ↓
读取 Authorization
  ↓
解析 JWT
  ↓
查询数据库用户
  ↓
比较 tokenVersion
  ↓
检查是否 BANNED
  ↓
request.setAttribute("currentUser", CurrentUserDTO)
  ↓
返回 true
  ↓
AccountController.getCurrentUser()
```

Fence 33

7.3 7.3 `@RequestAttribute` 从哪里拿数据

拦截器执行：

```
request.setAttribute(
        CURRENT_USER,
        new CurrentUserDTO(entity.getId(), entity.getAccount(),
entity.getRole()));
```

Fence 34

Controller 执行：

```
@RequestAttribute(AuthenticationInterceptor.CURRENT_USER)
CurrentUserDTO currentUserDTO
```

Fence 35

两边使用相同的名称 `currentUser`，因此 Controller 能取到拦截器放进去的对象。

这个 CurrentUserDTO 不是 Postman 在 Params 或 Body 中提交的。它只能由验证 Token 成功后的服务端代码产生，这样客户端不能冒充其他 userId。

7.4 7.4 为什么会收到“请先登录”

AuthenticationInterceptor 中：

```
if (authorization == null || !authorization.startsWith("Bearer ")) {
    throw unauthorized("请先登录");
}
```

Fence 36

常见原因：

- 请求没有 Authorization 头。
- 写成了 POST `/me`，但正确方法是 GET。
- Postman 登录成功后没有自动把 Token 带给下一请求。
- Bearer 后没有空格。
- Token 输入框带了双引号。
- 在 Postman Bearer Token 输入框中又手动写了 `Bearer`。

8. 8. 功能四：修改密码的完整链路

8.1 8.1 客户端请求

```
POST http://localhost:80/api/accounts/reset-password
Authorization: Bearer eyJhbGciOi...
Content-Type: application/json

{
  "currentPassword": "AAA123456",
  "newPassword": "AAA654321"
}
```

Fence 37

这个请求既需要 Token，也需要 JSON Body。

8.2 8.2 完整执行顺序

```
请求先经过 AuthenticationInterceptor
        ↓
Token 验证成功，产生 CurrentUserDTO
        ↓
Controller 接收 CurrentUserDTO 和 ResetPasswordDTO
```

```
        ↓
ServiceImpl 按 userId 查询数据库用户
        ↓
检查账号状态
        ↓
BCrypt 验证 currentPassword
        ↓
检查 newPassword 不能等于旧密码
        ↓
更新 password_hash
        ↓
token_version + 1
        ↓
更新 updated_at
        ↓
重新查询用户
        ↓
签发包含新 ver 的 JWT
        ↓
返回 LoginVO
```

Fence 38

8.3 8.3 为什么更新条件中还有旧 tokenVersion

```
.eq(UserAccountEntity::getId, entity.getId())
.eq(UserAccountEntity::getTokenVersion, oldVersion)
```

Fence 39

大致相当于：

```
UPDATE user_account
SET password_hash = ?, token_version = token_version + 1
WHERE id = ? AND token_version = ?;
```

Fence 40

如果两个修改密码请求同时到达，只有一个请求能使用原来的 tokenVersion 更新成功。另一个请求的更新行数会是 0，于是代码要求重新登录，避免两个请求互相覆盖。

这是一种简单的乐观锁思想。

8.4 8.4 修改密码后客户端必须做什么

修改密码响应会返回一个新 Token。Postman 或前端必须用新 Token 覆盖旧 Token。旧 Token 已因 tokenVersion 不一致而失效。

9. 9. DTO、Entity、VO 为什么不能混用

9.1 DTO：输入或内部传递

```
RegisterDTO：客户端允许提交的注册字段
CurrentUserDTO：后端内部传递的当前用户
```

Fence 41

9.2 Entity：数据库中的完整数据

UserAccountEntity 包含：

```
passwordHash
tokenVersion
status
role
```

Fence 42

Entity 不应该直接返回给前端，因为它含有敏感字段和数据库内部字段。

9.3 VO：允许输出的数据

AccountVO 不包含 passwordHash 和 tokenVersion，而且会脱敏：

```
demo@example.com  -> d***@example.com
13800138000       -> 138****8000
```

Fence 43

所以转换方向是：

```
请求 JSON -> DTO -> 业务 -> Entity -> VO -> 响应 JSON
```

Fence 44

10. 10. 必须认识的 Java 和 Spring 基本语法

10.1 10.1 `package` 和 `import`

```
package org.example.rebuiltzijingforum.account.controller;
```

Fence 45

表示这个类属于哪个包。

```
import org.example.rebuiltzijingforum.account.service.AccountService;
```

Fence 46

表示当前文件要使用其他包中的 AccountService。

10.2 10.2 类、接口、实现

接口只规定“能做什么”：

```
public interface AccountService {
    LoginVO login(LoginDTO loginDTO);
}
```

Fence 47

实现类负责“具体怎么做”：

```
public class AccountServiceImpl implements AccountService {
    @Override
    public LoginVO login(LoginDTO loginDTO) {
        // 实际逻辑
    }
}
```

Fence 48

- `implements`：实现一个接口。
- `@Override`：这个方法来自接口或父类。
- 接口让 Controller 依赖稳定的业务约定，而不是绑死具体实现。

10.3 10.3 构造器注入

```
private final AccountService accountService;

public AccountController(AccountService accountService) {
    this.accountService = accountService;
}
```

Fence 49

含义：AccountController 工作时需要 AccountService。Spring 创建 Controller 时，把可用的 AccountServiceImpl 传入构造器。

- `private`：只允许当前类使用。
- `final`：构造完成后不能换成另一个对象。
- `this.accountService`：当前对象的成员变量。
- 右侧 `accountService`：构造器接收到的参数。

10.4 10.4 泛型 `<T>`

```
ApiResponse<AccountVO>
```

Fence 50

可以理解为“data 部分装 AccountVO 的 ApiResponse”。

```
ApiResponse<LoginVO>
```

Fence 51

则表示 data 部分装 LoginVO。

`T` 是一个类型占位符，让同一个 ApiResponse 可以包装不同数据类型。

10.5 10.5 static 方法

```
ApiResponse.success(data)
AccountVO.from(entity)
```

Fence 52

`static` 方法属于类本身，不需要先 `new ApiResponse()` 或 `new AccountVO()` 才能调用。

10.6 10.6 方法引用 `::`

```
UserAccountEntity::getAccount
```

Fence 53

这里不是立即调用某个对象的 getAccount，而是把“UserAccountEntity 的 getAccount 方法”作为字段信息交给 MyBatis-Plus，用来生成数据库条件。

10.7 10.7 Lambda 和链式调用

```
new LambdaQueryWrapper<UserAccountEntity>()
        .eq(UserAccountEntity::getAccount, account)
        .last("LIMIT 1")
```

Fence 54

每次方法调用都返回当前条件对象，所以可以继续使用 `.eq(...).last(...)`，这叫链式调用。

10.8 10.8 异常

```
throw new BusinessException(...);
```

Fence 55

`throw` 表示当前流程不能继续，抛出错误。GlobalExceptionHandler 会把 BusinessException 转成对应状态码和 JSON。

```
try {
    userAccountMapper.insert(entity);
} catch (DataIntegrityViolationException exception) {
    throw conflict("账号、邮箱或手机号已存在");
}
```

Fence 56

- `try`：尝试执行。
- `catch`：捕获指定异常。
- 这里把数据库异常转换成用户更容易理解的业务错误。

10.9 10.9 常见 Spring 注解

注解	含义
`@RestController`	这是返回 JSON 的 Controller
`@RequestMapping`	定义 Controller 公共 URL 前缀
`@PostMapping`	接收 POST 请求
`@GetMapping`	接收 GET 请求
`@RequestBody`	从 JSON Body 创建 DTO
`@RequestAttribute`	从当前服务器请求属性中取对象

注解	含义
`@Valid`	执行 DTO 校验
`@Service`	把业务实现交给 Spring 管理
`@Component`	把普通组件交给 Spring 管理
`@Configuration`	这是 Spring 配置类
`@Bean`	方法返回值交给 Spring 管理
`@Transactional`	开启事务
`@Mapper`	这是 MyBatis Mapper
`@TableName`	Entity 对应哪个表
`@TableId`	Entity 哪个字段是主键

10.10 10.10 Lombok `@Data`

UserAccountEntity 使用：

```
1  @lombok.Data
```

Fence 57

Lombok 在编译时生成 getter、setter、toString、equals 和 hashCode，所以源代码中虽然看不到 `getId()`、`setAccount()`，仍然可以调用。

11. 11. Mapper 和数据库到底如何对应

UserAccountMapper：

```
1  public interface UserAccountMapper
2          extends BaseMapper<UserAccountEntity> {
3  }
```

Fence 58

BaseMapper 已经提供：

- `insert(entity)`：新增。
- `selectById(id)`：按主键查询。
- `selectOne(wrapper)`：按条件查询一条。
- `selectCount(wrapper)`：统计数量。
- `update(entity, wrapper)`：按条件更新。

因此本包暂时不需要手写 Mapper XML。

配置：

```yaml
mybatis-plus:
  configuration:
    map-underscore-to-camel-case: true
```

Fence 59

让数据库下划线字段和 Java 驼峰字段对应：

```
password_hash  <-> passwordHash
token_version  <-> tokenVersion
created_at     <-> createdAt
```

Fence 60

12. 12. H2 和 MySQL 的关系

默认启动时使用 H2 文件数据库：

```
E:/REBUILTZIJINGFORUM/data/rebuilt-zijing-forum.mv.db
```

Fence 61

启用 `mysql` profile 后才使用 application-mysql.yml 中的 MySQL。

```
默认启动      -> H2
mysql profile -> MySQL
```

Fence 62

H2 和 MySQL 是两个独立数据库，不会自动同步。你在 H2 注册的账号，不会自动出现在 MySQL。

用于查看 H2 数据的 SQL：

```sql
SELECT id,
       account,
       password_hash,
       email,
       mobile,
       role,
       status,
       token_version,
       created_at,
       updated_at
FROM public.user_account
ORDER BY id;
```

Fence 63

`password_hash` 只能看到哈希，不能看到原密码，这是正确的安全设计。

13. 13. ApiResponse 和异常响应

成功响应统一长这样：

```
{
  "success": true,
  "code": "OK",
  "message": "success",
  "data": {}
}
```

Fence 64

失败响应统一长这样：

```
{
  "success": false,
  "code": "INVALID_CREDENTIALS",
  "message": "账号或密码错误",
  "data": null
}
```

Fence 65

常见 HTTP 状态码：

状态码	含义	本包例子
200	成功	登录、查询、改密成功
201	创建成功	注册成功
400	请求参数错误	DTO 校验失败、当前密码错误
401	未认证	没 Token、Token 无效、登录失败
403	已认证但禁止操作	账号被禁用
409	数据冲突	账号、邮箱或手机号重复
500	未处理的服务器异常	当前统一异常处理的兜底响应

当前版本有一个需要知道的现象：错误 HTTP 方法、缺少 JSON Body 等 Spring MVC 异常还没有单独处理，可能被兜底包装成 500。正确设计通常应该把它们分别处理成 400、405、415。这个现象不代表注册业务真的写入了一半。

14. 14. 用 Postman 按顺序完成四个功能

14.1 第一步：注册

```
POST /api/accounts/register
Body -> raw -> JSON
```

Fence 66

确认返回 201。

14.2 第二步：登录

```
POST /api/accounts/login
Body -> raw -> JSON
```

Fence 67

复制 `data.accessToken`。只复制双引号内部以 `eyJ` 开头的内容。

14.3 第三步：查询本人

```
GET /api/accounts/me
Auth -> Bearer Token -> 粘贴 Token
Body -> none
```

Fence 68

14.4 第四步：修改密码

```
POST /api/accounts/reset-password
Auth -> Bearer Token -> 粘贴 Token
Body -> raw -> JSON
```

Fence 69

修改成功后复制响应中的新 Token。旧 Token 应当返回 TOKEN_REVOKED。

15. 15. 最常见的错误对照表

现象	原因	改法
注册返回 500	参数放在 Params，JSON Body 为空	放到 Body -> raw -> JSON
`/me` 返回 500	错用 POST	改成 GET
`/me` 返回“请先登录”	没有 Authorization 请求头	Auth 选择 Bearer Token
Token 无效	Token 带双引号	只粘贴 `eyJ...`
Token 无效	在 Token 输入框又写 `Bearer`	只粘贴 Token
登录成功但 `/me` 未登录	登录不会让 Postman 自动保存 Token	手动保存或使用环境变量
注册返回 409	相同账号、邮箱或手机已存在	换数据或直接登录
改密后旧 Token 失败	tokenVersion 已递增	使用改密响应的新 Token
H2 有数据但 MySQL 没有	当前只启用了 H2	明确选择一个 profile
IDEA Database 看不到表	元数据没有刷新	刷新并展开 public -> 表

16. 16. AccountApiTest 在验证什么

自动化测试会真实经过：

```
Controller
-> DTO 校验
-> ServiceImpl
-> Mapper
-> H2 内存数据库
-> Interceptor
-> JWT
-> 响应 JSON
```

Fence 70

主要验证：

1. 注册成功且数据库保存的是 BCrypt 哈希。
2. 登录成功并取得 Token。
3. 使用 Token 能查询当前用户。
4. 修改密码后得到新 Token。
5. 旧 Token 立即失效。
6. 旧密码不能再登录，新密码可以登录。
7. 重复注册返回 409。
8. 输入校验失败返回 400。
9. 没有 Token 返回 401。

测试使用 `src/test/resources/application.yml` 中的内存 H2，不会污染手工测试使用的文件 H2。每个测试前还会执行：

```
DELETE FROM user_account;
```

Fence 71

17. 17. 建议你自己动手做的五个练习

17.1 练习一：画注册链路

不看本文，自己写出：

```
Postman -> ? -> ? -> ? -> H2 -> ? -> JSON
```

Fence 72

17.2 练习二：观察密码哈希

注册一个账号，然后在 H2 查询 `password_hash`，确认它不是原密码。用相同原密码注册另一个账号时，两个 BCrypt 哈希通常也不同，这是因为 BCrypt 会加入随机盐。

17.3 练习三：观察账号标准化

使用大写账号注册，再查询数据库和响应，观察它为什么变成小写。

17.4 练习四：观察旧 Token 失效

保存改密前 Token，修改密码后分别用旧 Token 和新 Token 请求 `/me`，比较响应 code。

17.5 练习五：逐行跟踪

在以下位置打断点：

1. `AccountController.login()`。
2. `AccountServiceImpl.login()`。
3. `JwtUtils.createToken()`。
4. `AuthenticationInterceptor.preHandle()`。
5. `JwtUtils.parseToken()`。

使用 Postman 登录，再请求 `/me`，观察执行顺序。

18. 18. 学完后应该能回答的问题

如果下面问题能用自己的话回答，F01 就真正理解了：

1. 为什么 Params 不能代替 `@RequestBody` JSON?
2. 为什么 Entity 不能直接返回给前端?
3. 为什么密码只能验证，不能从哈希还原?
4. 为什么 Java 查重后数据库还要 UNIQUE?
5. 为什么登录成功后 `/me` 仍必须带 Token?
6. JWT 的三段分别是什么?
7. JWT 为什么不是加密?
8. `sub`、`ver`、`iat`、`exp` 各是什么?
9. 拦截器如何把 CurrentUserDTO 交给 Controller?
10. 为什么修改密码后旧 Token 能立刻失效?
11. `@Transactional` 解决什么问题?
12. H2 和 MySQL 为什么不会同时出现相同数据?

19. 19. 最后用一句话概括四条链路

```
注册：校验输入 -> 标准化 -> 查重 -> BCrypt -> INSERT -> 脱敏返回

登录：查询账号 -> BCrypt matches -> 检查状态 -> 生成 JWT -> 返回 Token

当前用户：Bearer Token -> 拦截器验签/验过期/验版本 -> 放入 CurrentUserDTO -> 查询并返回

修改密码：验证 Token -> 验证旧密码 -> 更新哈希和 tokenVersion -> 生成新 JWT -> 旧 JWT 失效
```